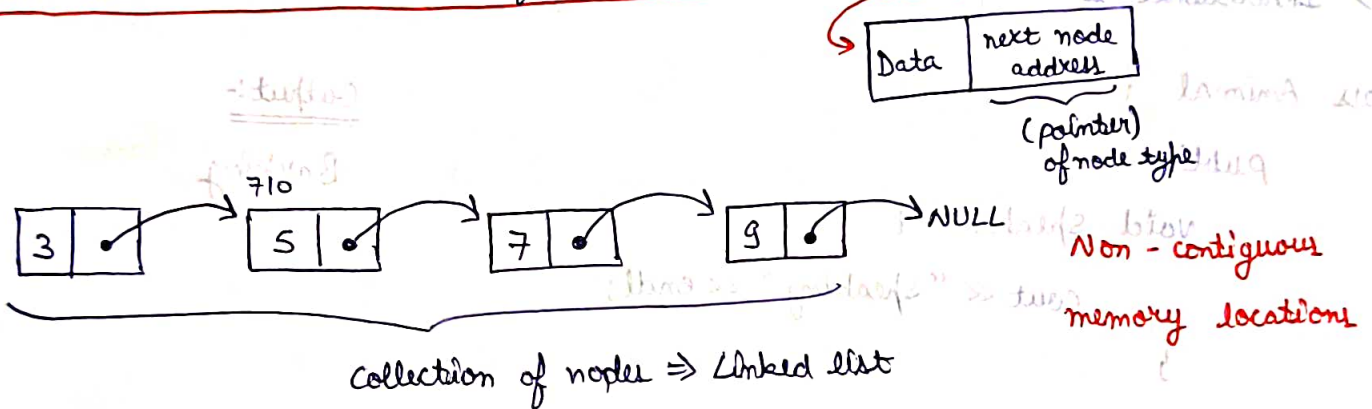


Lecture (44)

Linked List

↳ linear data structure made from collection of nodes

Node

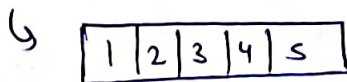


Why?

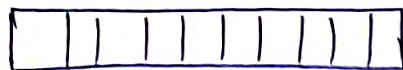
$\Rightarrow$ arr [10] ka kya mai 100 size ka bana sakta hu run time me $\Rightarrow$ Nahi

$\Rightarrow$ In Vector (Dynamic Array)

↳ `vector<int> v(5);`



size 10 $\rightarrow$



{ But this is not optimal }
① new storage create
② values copy

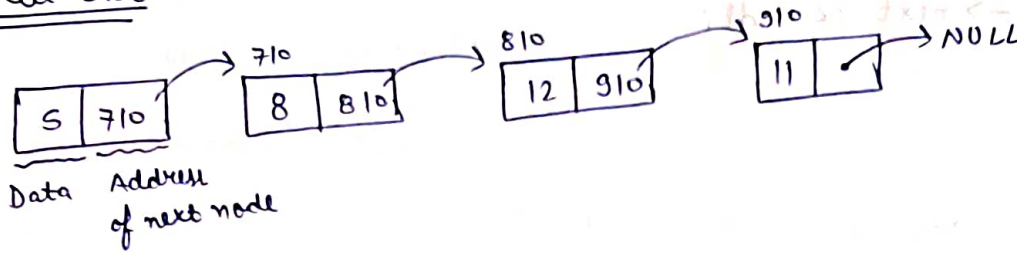
Therefore Linked list

Dynamic Data Structure

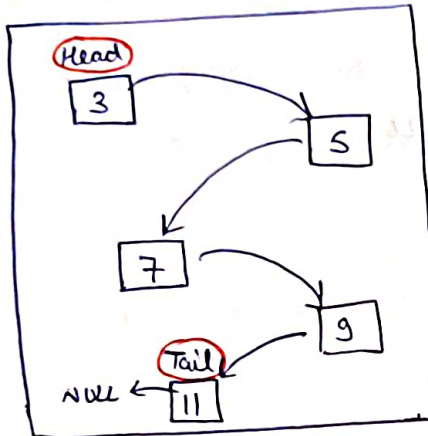
⇒ can be grow/shrink in run time so no memory wastage

⇒ Insertion/Deletion is easy because no shift needed

Linked List



Heap



No need of contiguous location

We must never lose the address of the head pointer as it references the starting address of the linked list and if lost, would lead to loss of the list.

Types of Linked List

- Singly LL
- Doubly LL
- Circular LL
- Circular Doubly LL

Singly Linked List

Making Node Class

```
class Node {
```

```
public:
```

```
    int data;
```

```
    Node * next;
```

```
    Node(int data) {
```

```
        this->data = data;
```

```
        this->next = NULL;
```

```
    }
```

```
};
```

Constructor

```

int main() {
    Node * node1 = new Node(10);
    cout << node1 -> data << endl;
    cout << node1 -> next << endl;

    return 0;
}

```

Output:-

10
0x0

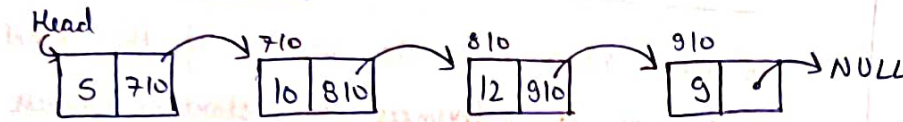
this is the output

```

graph LR
    node1[Node] --> data[10]
    node1 --> next[0x0]

```

Singly Linked list



Insert at Head / Tail / Any position

```

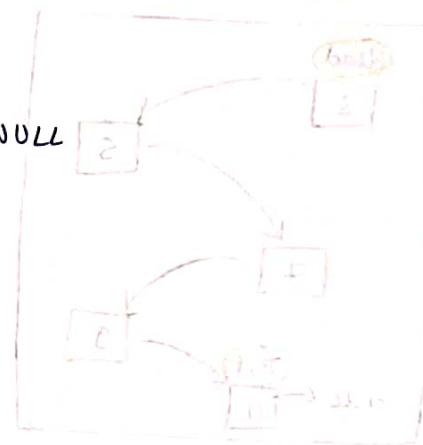
class Node {
public:
    int data;
    Node * next;

    // Constructor
    Node(int data) {
        this -> data = data;
        this -> next = NULL;
    }

    // destructor
    ~Node() {
        int value = this -> data;

        // memory free
        if (this -> next != NULL) {
            delete next;
            this -> next = NULL;
        }
    }
}

```



this is the code for insert at head

```

1) Node * temp = head;
2) Node * newnode = new Node(data);
3) newnode -> next = temp;
4) head = newnode;

```

Insert at Tail

Head should be null

if head is null

data is

temp = head

if (data != null) {

temp = temp -> next;

temp = data -> next;

}

;

```

void insertAtHead (Node* &head, int d) {
    //new node create
    Node* temp = new Node(d);
    temp->next = head;
    head = temp;
}

```

```

void insertAtTail (Node* &tail, int d) {
    //new node create
    Node* temp = new Node(d);
    tail->next = temp;
    tail = temp;
}

```

```

void insertAtPosition (Node* &tail, Node* &head, int position, int d) {
    //insert at start
    if (position == 1) {
        insertAtHead(head, d);
        return;
    }
    Node* temp = head;
    int cnt = 1;
    while (cnt < position - 1) {
        temp = temp->next;
        cnt++;
    }
    //inserting at last position
    if (temp->next == NULL) {
        insertAtTail(tail, d);
        return;
    }
}

```


// creating a node for d { (b, true, head & *head) head & *head

Node* nodeToInsert = new Node(d);

nodeToInsert -> next = temp -> next; cur = nodeToInsert

temp -> next = nodeToInsert; head = cur -> next

} if not = head

void deleteNode(int position, Node* &head) {

// deleting first or start node

if (position == 1) {

Node* temp = head;

head = head -> next;

// memory free start node

temp -> next = NULL;

delete temp;

}

else {

// deleting any middle node or last node

Node* cur = head;

Node* prev = NULL;

int cnt = 1;

while (cnt < position) {

prev = cur; cur = cur -> next;

cur = cur -> next; ++cnt

cnt++;

}

prev -> next = cur -> next;

cur -> next = NULL;

delete cur;

}

}

```
void print (Node * head) {
```

```
    if (head == NULL) {
```

```
        cout << "List is empty" << endl;
```

```
        return;
```

```
    }
```

```
    Node * temp = head;
```

```
    while (temp != NULL) {
```

```
        cout << temp->data << " ";
```

```
        temp = temp->next;
```

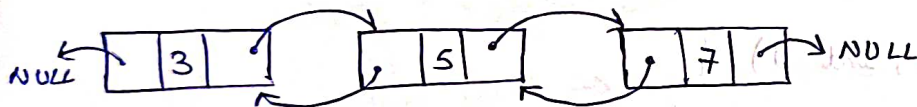
```
    }
```

```
    cout << endl;
```

```
}
```

Doubly Linked List

Node



```
class Node {
```

```
public:
```

```
    int data;
```

```
    Node * prev;
```

```
    Node * next;
```

```
// Constructor
```

```
Node (int d) {
```

```
    this->data = d;
```

```
    this->prev = NULL;
```

```
    this->next = NULL;
```

```
}
```

```
};
```

```
void print (Node * head) {
```

```
    Node * temp = head;
```

```
    while (temp != NULL) {
```

```
        cout << temp->data << " ";
```

```
        temp = temp->next;
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
int getLength (Node * head) {
```

```
    int len = 0;
```

```
    Node * temp = head;
```

```
    while (temp != NULL) {
```

```
        len++;
```

```
        temp = temp->next;
```

```
    }
```

```
    return len;
```

```
}
```

```
void insertAtHead (Node * &head, int d) {
```

```
    Node * temp = new Node(d);
```

```
    temp->next = head;
```

```
    head->prev = temp;
```

```
    head = temp;
```

```
}
```

```
void insertAtTail (Node * &tail, int d) {
```

```
    Node * temp = new Node(d);
```

```
    tail->next = temp;
```

```
    temp->prev = tail;
```

```
    tail = temp;
```

```
}
```



```

void insertAtPosition (Node * &head, int position, int d, Node * &tail) {
    // Insert at start
    if (position == 1) {
        insertAtHead (head, d);
        return;
    }

    Node * temp = head;
    int cnt = 1;
    while (cnt < position - 1) {
        temp = temp -> next;
        cnt++;
    }

    // Inserting at last position
    if (temp -> next == NULL) {
        insertAtTail (tail, d);
        return;
    }

    Node * nodeToInsert = new Node(d);
    nodeToInsert -> next = temp -> next;
    temp -> next -> prev = nodeToInsert;
    temp -> next = nodeToInsert;
    nodeToInsert -> prev = temp;
}

```

Correction

for empty list

```

if (head == NULL) {
    Node * temp = new Node(d);
    head = temp;
}

```

In insertAtHead() for similarly, for tail

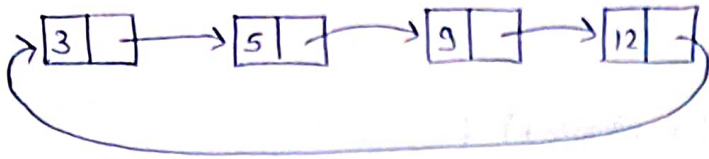

```

void deleteNode (int position, Node* &head) {
    // deleting first or start node
    if (position == 1) {
        Node * temp = head;
        temp -> next -> prev = NULL;
        head = temp -> next;
        temp -> next = NULL;
        delete temp;
    }
    else {
        // deleting any middle node or last node
        Node * curr = head;
        Node * prev = NULL;
        int cnt = 1;
        while (cnt < position) {
            prev = curr;
            curr = curr -> next;
            cnt++;
        }
        curr -> prev = NULL;
        prev -> next = curr -> next;
        curr -> next = NULL;
        delete curr;
    }
}

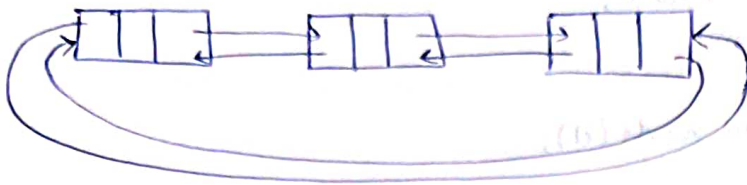
```

Destructor: Bhi likhna padega same as singly linked list

Circular Linked List



No need of head in this because we can access first position by tail.



```
class Node {
```

```
public:
```

```
int data;
```

```
Node * next;
```

```
// Constructor
```

```
Node (int d) {
```

```
    this->data = d;
```

```
    this->next = NULL;
```

```
}
```

```
~Node () {
```

```
    int value = this->data;
```

```
    if (this->next != NULL) {
```

```
        delete next;
```

```
        next = NULL;
```

```
    }
```

```
}
```

```
};
```

```
void insertNode (Node* &tail, int element, int d) {
```

```
    // assuming that the element is present in the list
```

```
    // Empty list
```

```
    if (tail == NULL) {
```

```
        Node * newNode = new Node(d);
```

```
        tail = newNode;
```

```
        newNode->next = newNode;
```

```
    }
```

else {

// Non-empty list

Node *curr = tail;

while (curr -> data != element) {

curr = curr -> next;

}

Node *temp = new Node(0);

temp -> next = curr -> next;

curr -> next = temp;

}

}

void print (Node *tail) {

Node *temp = tail;

~~while (temp != NULL)~~

do {

cout << tail -> data << " ";

tail = tail -> next;

} while (tail != temp)

cout << endl;

}

void deleteNode (Node * &tail, int value) {

if (tail == NULL) {

cout << "List is empty" << endl;

return;

}

else {

Node *prev = tail;

Node *curr = prev -> next;

while (curr -> data != value) {

```

    prev = curr;
    curr = curr → next;
}
prev → next = curr → next;
curr → next = NULL;
delete curr;
}

```

```

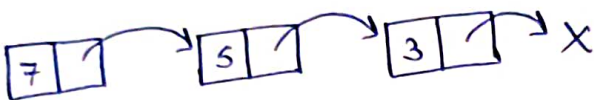
if (curr == prev) {
    tail = NULL;
}
if (tail == curr) {
    tail = prev;
}

```

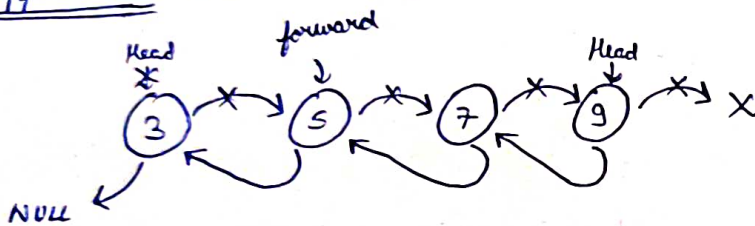
Lecture 45

Qn - Reverse a linked list

i/p ⇒ 

o/p ⇒ 

Approach ① (Iterative way)



```
Node * prev = NULL;
```

```
Node * curr = head;
```

forward pointer lagane se pehle hi agar pichla waala kat diya To

poori linked list lost Ho jayegi

```
while (curr != NULL) {
    forward = curr → next;
```

```
    curr → next = prev;
```

```
    prev = curr;
```

```
    curr = forward;
```

```
}
```

```
return prev;
```

} Ye reversed linked list ka Head return karega

Code:-

```
Node * reverseLinkedList(Node * head) {
```



```
if (head == NULL || head->next == NULL) {
```

```
    return head;
```

```
}
```

```
Node *prev = NULL;
```

```
Node *curr = head;
```

```
Node *forward = NULL;
```

```
while (curr != NULL) {
```

```
    forward = curr->next;
```

```
    curr->next = prev;
```

```
    prev = curr;
```

```
    curr = forward;
```

```
}
```

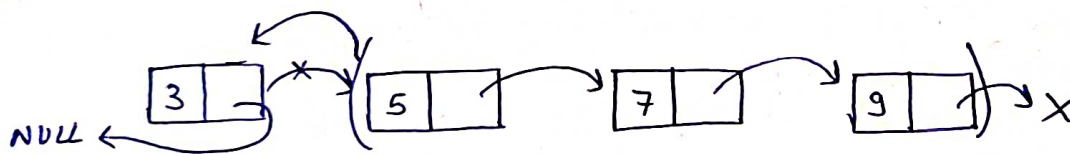
```
return prev;
```

```
}
```

T.C. $\Rightarrow O(n)$

S.C. $\Rightarrow O(1)$

Approach ② (Recursive way)



1 case solve Karlo baaki recursion dekh lega

```
void reverse(Node *head, Node *curr, Node *prev) {
```

```
    if (curr == NULL) {
```

```
        head = prev;
```

```
        return;
```

```
}
```

```
Node *forward = curr->next;
```

```
reverse(head, forward, curr);
```

```
curr->next = prev;
```

```
}
```

```
Node * reverseLinkedList (Node * head) {
```

```
    Node * curr = head;
```

```
    Node * prev = NULL;
```

```
    reverse (head, curr, prev);
```

```
    return head;
```

```
}
```

T.C = $O(n)$

S.C = $O(n)$

Approach ③

```
Node * reverse (Node * head) {
```

```
    if (head == NULL || head->next == NULL) {
```

```
        return head;
```

```
    }
```

```
    Node * chotaHead = reverse (head->next);
```

```
    head->next->next = head;
```

```
    head->next = NULL;
```

```
    return chotaHead;
```

```
}
```

T.C = $O(n)$

S.C = $O(n)$

H.W.

Reverse Doubly Linked list

Q- Middle of linked list

i/p $\Rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

o/p $\Rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Approach ①

length of linked list

mid = $\frac{\text{len}}{2}$ (even)

mid = $\frac{\text{len}}{2} + 1$ (odd)

(mid-1) / 2

mid

low = 0, high = 5

mid = 3

mid = 3

```

int getLength(Node *head) {
    int len = 0;
    while (head != NULL) {
        len++;
        head = head -> next;
    }
    return len;
}

```

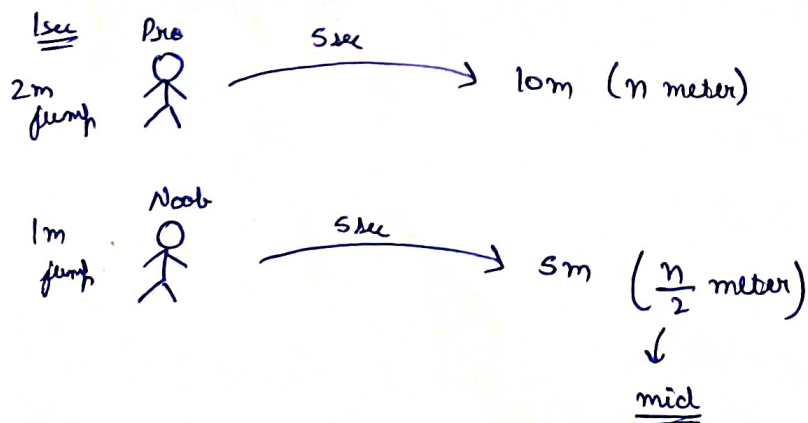
```

Node *findMiddle(Node *head) {
    int len = getLength(head);
    int ans = (len/2);

    Node *temp = head;
    int cnt = 0;
    while (cnt < ans) {
        temp = temp -> next;
        cnt++;
    }
    return temp;
}

```

Optimised solⁿ



(head & temp) tell both is same & stop
 head = head -> next
 temp = temp -> next
 (head & temp) tell both is same & stop
 head = head -> next

T.C. = $O(n)$

(also) $\frac{mid}{1} = \text{time}$

(also) $1 + \frac{mid}{1} = \text{time}$

empty list $\rightarrow$ return NULL

1 node $\rightarrow$ (head)

2 node $\rightarrow$ (head $\rightarrow$ next)

Node *getMiddle (Node *head) {

if (head == NULL || head $\rightarrow$ next == NULL) {

return head;

}

if (head $\rightarrow$ next $\rightarrow$ next == NULL) {

return head $\rightarrow$ next;

}

Node *slow = head;

Node *fast = head $\rightarrow$ next;

while (fast != NULL) {

fast = fast $\rightarrow$ next;

if (fast != NULL) {

fast = fast $\rightarrow$ next;

}

slow = slow $\rightarrow$ next;

}

return slow;

}

Lecture 46

Qn- Reverse Linked list (K-groups)

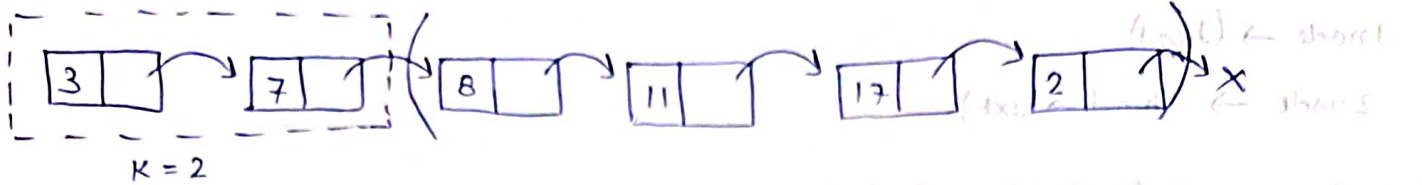
i/p $\Rightarrow$ 
K=2

o/p $\Rightarrow$ 

(n) O = 0.7

(n) O = 0.2

Approach



```
Node * kReverse (Node * head, int K) {
```

```
// Base case
```

```
if (head == NULL) {
```

```
    return NULL;
```

```
}
```

```
// step 1: reverse first K nodes
```

```
Node * next = NULL;
```

```
Node * curr = head;
```

```
Node * prev = NULL;
```

```
int count = 0;
```

```
while (curr != NULL && count < K) {
```

```
    next = curr -> next;
```

```
    curr -> next = prev;
```

```
    prev = curr;
```

```
    curr = next;
```

```
    count++;
```

```
}
```

```
// step 2: Recursion dekh lega aage ka
```

```
if (next != NULL) {
```

```
    head -> next = kReverse(next, K);
```

```
}
```

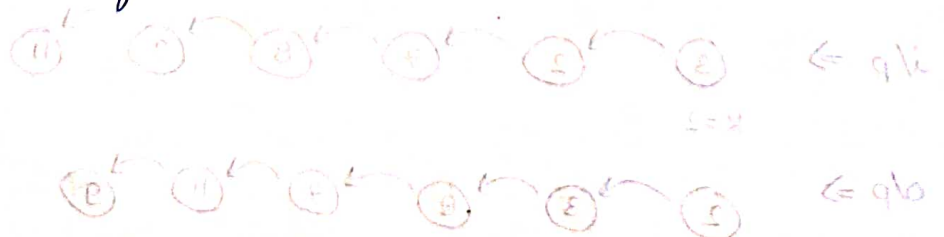
```
// step 3: return head of reversed list
```

```
return prev;
```

```
}
```

T.C. = $O(n)$

S.C. = $O(n)$



Ques- Check whether the linked list is circular or not

bool isCircularList(Node* head) {

if (head == NULL) {

return ~~false~~ true;

}

Node* temp = head->next;

while (temp != NULL && temp != head) {

temp = temp->next;

}

if (temp == head) {

return true;

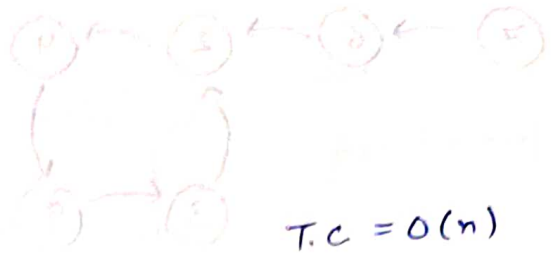
}

if (temp == NULL) {

return false;

}

}



T.C = $O(n)$

S.C. = $O(1)$

H.W.

Floyd's Cycle detection algorithm

Lecture 47

Ques- Detect and Remove loop from linked list and also return the beginning node of loop in linked list.

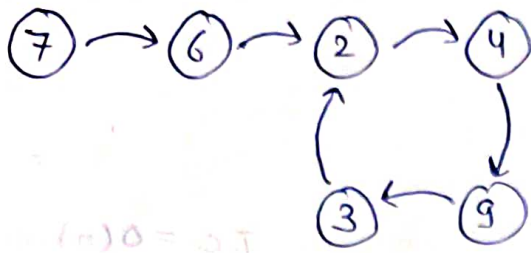
Solⁿ

This question can be broken into three parts

- Detect cycle in L.L.
- Remove cycle from L.L.
- Beginning node of loop in L.L.



Approach



① detect cycle $\rightarrow$ T/F

map create $\Rightarrow$ map < $\frac{\text{Node}^*}{\text{Key}}$, $\frac{\text{bool}}{\text{value}}$ > visited;

bool detectLoop (Node* head) {

if (head == NULL) {

return false;

}

map < Node*, bool > visited;

Node* temp = head;

while (temp != NULL) {

// cycle is present

if (visited[temp] == true) {

cout << "Present on element " << temp->data << endl;

return true;

}

visited[temp] = true;

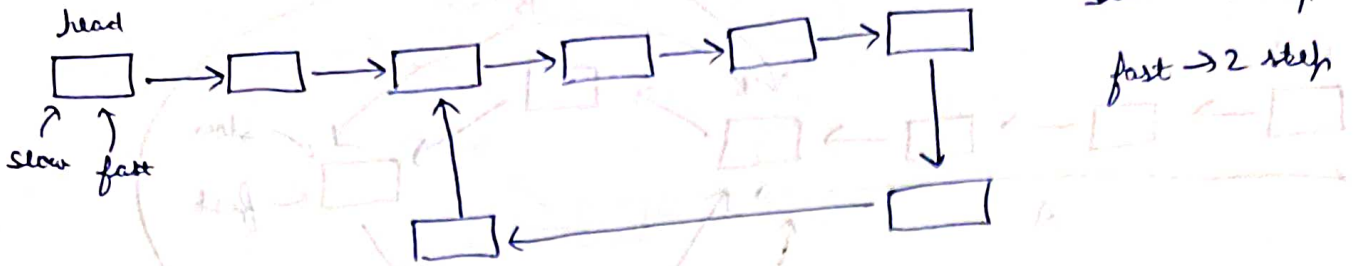
temp = temp->next;

return false;

}



Floyd's cycle detection algorithm



slow $\rightarrow$ 1 step

fast $\rightarrow$ 2 steps

if (slow == fast)
 $\hookrightarrow$ loop is present

if (fast == NULL)
 $\hookrightarrow$ no loop

Node * floydDetectLoop (Node * head) {

if (head == NULL) {
 return NULL;
}

Node * slow = head;

Node * fast = head;

while (slow != NULL && fast != NULL) {

fast = fast $\rightarrow$ next;

if (fast != NULL) {

fast = fast $\rightarrow$ next;

}

slow = slow $\rightarrow$ next;

if (slow == fast) {

cout << "Present at " << slow $\rightarrow$ data << endl;

return slow;

}

}

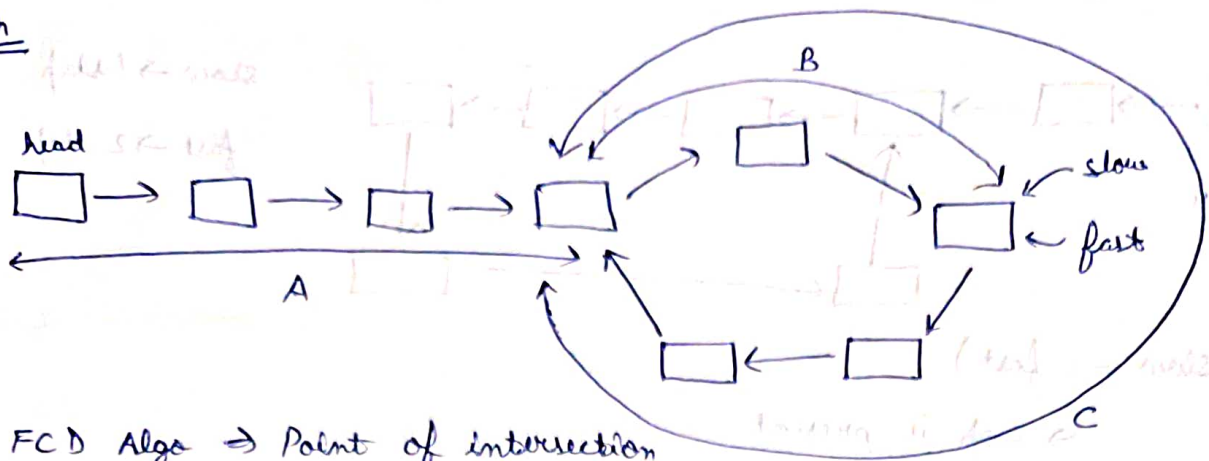
return NULL;

T.C. $\Rightarrow O(n)$

S.C. $\Rightarrow O(1)$

Now finding starting node of loop

Approach



① FCD Algo $\Rightarrow$ Point of intersection

② slow = head, & move both pointers by same pace

③ when (slow == fast) $\Rightarrow$ starting point of loop

Distance by fast pointer = 2 * distance by slow pointer

$$A + x * C + B = 2 * (A + y * C + B)$$

$$A + xC + B = 2A + 2yC + 2B$$

$$A + B = C(x - 2y)$$

$$\text{let } x - 2y = K$$

$$\Rightarrow A + B = K \text{ times } C$$

$$\text{So, } \underline{A + B = \text{Cycle complete}}$$

Node * getStartingNode(Node * head) {

if (head == NULL) {

return NULL;

}

Node * intersection = floydDetection(head);

Node * slow = head;

while (slow != intersection) {

slow = slow -> next;

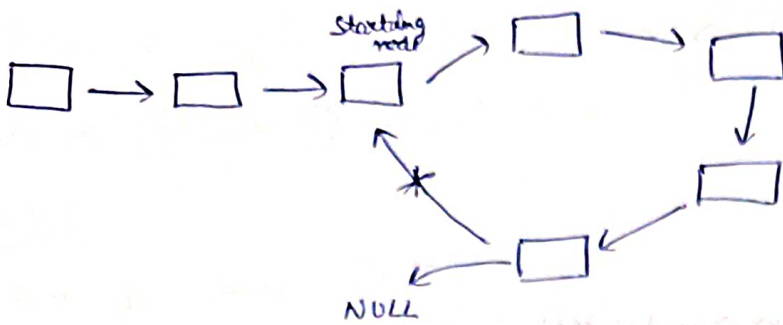
intersection = intersection -> next;

}

return slow;

}

Now Remove loop from LL



```
void removeLoop (Node * head) {
    if (head == NULL) {
        return ;
    }
```

```
    Node * startofLoop = getStartingNode (head);
```

```
    Node * temp = startofLoop;
```

```
    while (temp->next != startofLoop) {
```

```
        temp = temp->next;
```

```
    }
```

```
    temp->next = NULL;
```

```
}
```

H.W.

Remove loop → 5 ways

Detect cycle → 3 ways

} Read & code

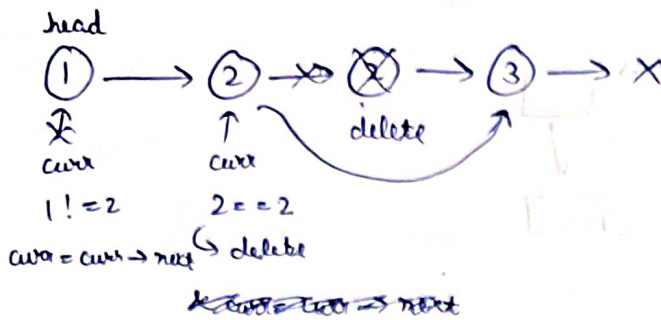
Lecture 48

Ques- Remove duplicates from a sorted L.L.

I/P ⇒ ① → ② → ② → ③ → ③ → ③ → ③ → ④ → X

O/P ⇒ ① → ② → ③ → ④ → X

Approach



① $curr \rightarrow next = curr \rightarrow next \rightarrow next$

② delete duplicate

Code

`Node* uniqueSortedList (Node* head) {`

`if (head == NULL) {`

`return NULL;`

`}`

`Node* curr = head;`

`while (curr != NULL) {`

`if (curr->data == curr->next->data && curr->next != NULL) {`

`Node* next-next = curr->next->next;`

`Node* nodeToDelete = curr->next;`

`delete (nodeToDelete);`

`curr->next = next-next;`

`}`

`else {`

`curr = curr->next;`

`}`

`}`

`return head;`

`}`

T.C. = $O(n)$

S.C. = $O(1)$

Ques- Remove Duplicates from unsorted L.L.

I/P $\Rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 2 \rightarrow X$

O/P $\Rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow X$

Approach I

Two for loops $\Rightarrow T.C = O(n^2)$
S.C = $O(1)$

curr = head;

while (curr != NULL) {

Node * temp = curr -> next;

while (temp != NULL) {

if (curr -> data == temp -> data) {

delete

}

}

}

Approach II

① Sort L.L. $\rightarrow O(n \log n)$

② Use previous algo $\rightarrow O(n)$

So, T.C = $O(n \log n)$

S.C = $O(1)$

Approach III

map < Node*, bool > visited;

(Using map)

$\hookrightarrow T.C = O(n)$

S.C = $O(n)$

H.W.

① Code previous question

② Split circular L.L. in 2 halves

Lecture 49

Ques- Sort 0, 1, 2 in L.L

i/p $\Rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow X$

o/p $\Rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow X$

Approach 1

Count 0, 1, 2

no	count
0	1
1	2
2	2

Now place

Code:-

```
Node* sortList(Node* head) {
```

```
    int zeroCount = 0;
```

```
    int oneCount = 0;
```

```
    int twoCount = 0;
```

```
    Node* temp = head;
```

```
    while (temp != NULL) {
```

```
        if (temp->data == 0)
```

```
            zeroCount++;
```

```
        else if (temp->data == 1)
```

```
            oneCount++;
```

```
        else if (temp->data == 2)
```

```
            twoCount++;
```

```
        temp = temp->next;
```

```
    }
```

temp = head;

while (temp != NULL) {

if (zeroCount != 0) {

temp -> data = 0;

zeroCount--;

}

else if (oneCount != 0) {

temp -> data = 1;

oneCount--;

}

else if (twoCount != 0) {

temp -> data = 2;

twoCount--;

}

temp = temp -> next;

}

return head;

}

T.C. = O(n)

Approach ②

↳ Data replacement is not allowed

so, we can change links

Create 3 L.L. { zero L.L.
one L.L. ⇒ Then merge
Two L.L.

Node* sortList (Node* head) {

Node* zeroHead = new Node(-1);

Node* oneHead = new Node(-1);

Node* twoHead = new Node(-1);

} Creating Dummy Node

Node* zeroTail = zeroHead;

Node* oneTail = oneHead;

Node* twoTail = twoHead;

Node* curr = head;

while (curr != NULL) {

int value = curr->data;

if (value == 0) {

populate(zeroTail, curr);

}

else if (value == 1) {

populate(oneTail, curr);

}

else if (value == 2) {

populate(twoTail, curr);

}

curr = curr->next;

}

if (oneHead->next != NULL) {

zeroTail->next = oneHead->next;

}

else {

zeroTail->next = twoHead->next;

}

oneTail->next = twoHead->next;

twoTail->next = NULL;

head = front

if (front == NULL) return

if (curr == NULL) return

curr = curr->next

return

populate is just an insert at Tail function

populate () {

tail->next = curr;

tail = curr;

}

⑤ Approach

we also have to increment the tail

while (curr != NULL) {

if (curr->data == 0) {
zeroTail->next = curr;
zeroTail = curr;
}

if (curr->data == 1) {

oneTail->next = curr;
oneTail = curr;
}

if (curr->data == 2) {
twoTail->next = curr;
twoTail = curr;
}

head = zeroHead → next;

delete zeroHead;

delete oneHead;

delete twoHead;

return head;

}

T.C ⇒ O(n)

S.C ⇒ O(1)

Qu- Merge two sorted Linked List

i/p ⇒ Head 1
① → ④ → ⑤ → X

Head 2
② → ③ → ⑤ → X

o/p ⇒ ① → ② → ③ → ④ → ⑤ → ⑤ → X

Approach

if (head1 == NULL) {

return head2;

}

if (head2 == NULL) {

return head1;

}

Code:-

Node<int>* sortTwoLists (Node<int>* first, Node<int>* second) {

if (first == NULL) {

return second;

}

if (second == NULL) {

return first;

}


```
if (first->data <= second->data) {
```

```
    solve (first, second);
```

```
}
```

```
else {
```

```
    solve (second, first);
```

```
}
```

```
}
```

Solve fn

Node<int> *

```
Node solve (Node<int> * first, Node<int> * second) {
```

```
    Node<int> * cur1 = first;
```

```
    Node<int> * next1 = cur1->next;
```

```
    Node<int> * cur2 = second;
```

```
    Node<int> * next2 = cur2->next;
```

```
    while (next1 != NULL && cur2 != NULL) {
```

```
        if ((cur2->data >= cur1->data) && (cur2->data <= next1->data)) {
```

```
            cur1->next = cur2;
```

```
            next2 = cur2->next;
```

```
            cur2->next = next1;
```

```
            cur1 = cur2;
```

```
            cur2 = next2;
```

```
        else {
```

```
            cur1 = next1;
```

```
            next1 = next1->next;
```

```
            if (next1 == NULL) {
```

```
                cur1->next = cur2;
```

```
                return first;
```

```
}
```

return first;

Lecture 50

Q- Check if the linked list is palindrome or not

Approach 1

- 1) Create an array
- 2) Copy L.L. content into array
- 3) Write logic to check palindrome

S.C. $\Rightarrow O(n)$

T.C. $\Rightarrow O(n)$

bool isPalindrome(Node *head) {

Vector<int> arr;

Node *temp = head;

while (temp != NULL) {

arr.push_back(temp->data);

temp = temp->next;

}

int n = arr.size();

int s = 0;

int e = n - 1;

while (s <= e) {

if (arr[s] != arr[e]) {

return 0;

}

s++;

e--;

}

return 1;

}

Approach 2



① Find middle

T.C $\Rightarrow O(n)$

② Reverse L.L. after middle

S.C $\Rightarrow O(1)$



③ Compare both halves

④ Repeat 2nd step

```
Node* getMid (Node* head) {
```

```
    Node* slow = head;
```

```
    Node* fast = head  $\rightarrow$  next;
```

```
    while (fast  $\neq$  NULL && fast  $\rightarrow$  next  $\neq$  NULL) {
```

```
        fast = fast  $\rightarrow$  next  $\rightarrow$  next;
```

```
        slow = slow  $\rightarrow$  next;
```

```
    }
```

```
    return slow;
```

```
}
```

```
bool isPalindrome (Node* head) {
```

```
    if (head  $\rightarrow$  next == NULL) {
```

```
        return true;
```

```
    }
```

```
    Node* middle = getMid (head);
```

```
    Node* temp = middle  $\rightarrow$  next;
```

```
    middle  $\rightarrow$  next = reverse (temp);
```

```
    Node* head1 = head;
```

```
    Node* head2 = middle  $\rightarrow$  next;
```

```
while (head2 != NULL) {
```

```
    if (head1->data != head2->data) {
```

```
        return false;
```

```
    }
```

```
    head1 = head1->next;
```

```
    head2 = head2->next;
```

```
}
```

```
temp = middle->next;
```

```
middle->next = reverse(temp);
```

```
return true;
```

```
}
```

```
Node* reverse(Node* head) {
```

```
    Node* curr = head;
```

```
    Node* prev = NULL;
```

```
    Node* next = NULL;
```

```
    while (curr != NULL) {
```

```
        next = curr->next;
```

```
        curr->next = prev;
```

```
        prev = curr;
```

```
        curr = next;
```

```
    }
```

```
    return prev;
```

```
}
```

Lecture 51

Ques -> Add 2 no's represented by L.L.

-> Add 1 to a no's represented by L.L.

i/p $\Rightarrow$ $^a \rightarrow$ (3) $\rightarrow$ (4) $\rightarrow$ X

$^b \rightarrow$ (2) $\rightarrow$ (3) $\rightarrow$ (0) $\rightarrow$ X

O/p $\Rightarrow$ (2) $\rightarrow$ (6) $\rightarrow$ (4) $\rightarrow$ X

Approach

i/p $\Rightarrow$ ^{Head 1} (4) $\rightarrow$ (5) $\rightarrow$ X

^{Head 2} (3) $\rightarrow$ (4) $\rightarrow$ (5) $\rightarrow$ X

^{ans} O/p $\Rightarrow$ (3) $\rightarrow$ (9) $\rightarrow$ (0) $\rightarrow$ X

$$\begin{array}{r} 45 \\ + 345 \\ \hline 390 \end{array}$$

Intuition

$\hookrightarrow$ reverse L.L.

$^f \rightarrow$ (5) $\rightarrow$ (4) $\rightarrow$ X

$^s \rightarrow$ (5) $\rightarrow$ (4) $\rightarrow$ (3) $\rightarrow$ X

^{ans} $\rightarrow$ (0) $\rightarrow$ (9) $\rightarrow$ (3) $\rightarrow$ X

reverse ans $\Rightarrow$ (3) $\rightarrow$ (9) $\rightarrow$ (0) $\rightarrow$ X

Node* reverse(Node* head) {

Node* curr = head;

Node* prev = NULL;

Node* next = NULL;

while (curr != NULL) {

next = curr $\rightarrow$ next;

curr $\rightarrow$ next = prev;

prev = curr;

curr = next;

}

return prev;

② initial

```
struct Node* addTwoLists (struct Node* first, struct Node* second) {
```

```
// step 1 - reverse input L.L.
```

```
first = reverse (first);
```

```
second = reverse (second);
```

```
// step 2 - add 2 L.L.
```

```
Node* ans = add (first, second);
```

```
// step 3 - reverse ans
```

```
ans = reverse (ans);
```

```
return ans;
```

```
}
```

Code for add:-

```
struct Node* add (struct Node* first, struct Node* second) {
```

```
int carry = 0;
```

```
Node* ansHead = NULL;
```

```
Node* ansTail = NULL;
```

```
while (first != NULL && second != NULL) {
```

```
int sum = carry + first->data + second->data;
```

```
int digit = sum % 10;
```

```
Node* temp = new Node (digit);
```

```
insertAtTail (ansHead, ansTail, digit);
```

```
carry = sum / 10;
```

```
first = first->next;
```

```
second = second->next;
```

```
}
```

```
while (first != NULL) {
```

```
int sum = carry + first->data;
```

```
int digit = sum % 10;
```

```
insertAtTail (ansHead, ansTail, digit);
```

```
carry = sum / 10;
```

```
first = first->next;
```

```
}
```

while (second != NULL) {

int sum = carry + second->data;

int digit = sum % 10;

insertAtTail (ansHead, ansTail, digit);

carry = sum / 10;

second = second->next;

}

while (carry != 0) {

int sum = carry;

int digit = sum % 10;

insertAtTail (ansHead, ansTail, digit);

carry = sum / 10;

}

return ansHead;

}

void insertAtTail (struct Node* &head, struct Node* &tail, int val) {

Node* temp = new Node(val);

if (head == NULL) {

head = temp;

tail = temp;

return;

}

else {

tail->next = temp;

tail = temp;

}

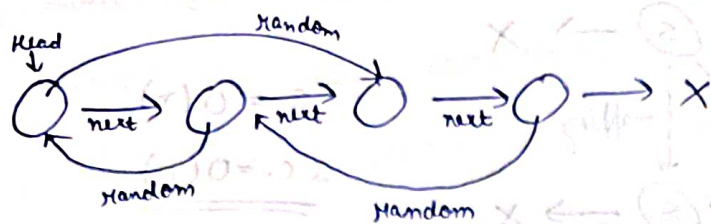
}

T.C $\Rightarrow O(m+n)$

S.C $\Rightarrow O(\max(m,n))$

Lecture 52

Qn- Clone a linked list with random and next pointer



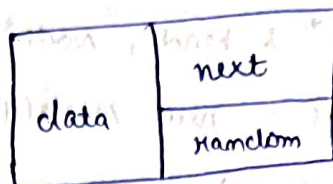
i.e. Node {

int data;

Node* next;

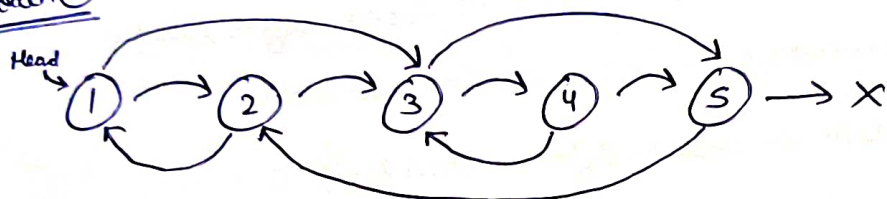
Node* random;

}



Node

Approach 1



① Create a clone list (using original list next pointer) $\rightarrow O(n)$



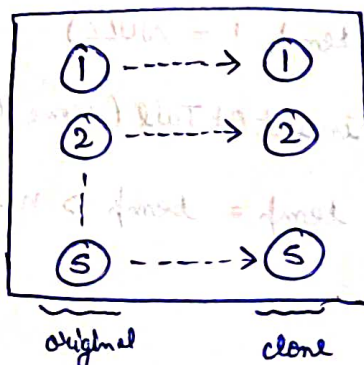
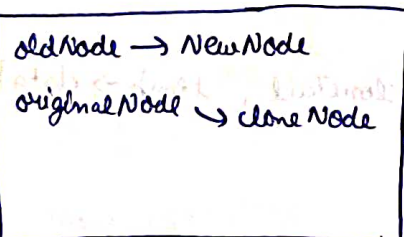
② Random pointer copy? $\rightarrow O(n^2)$ (Original list me count karke)

Approach 2

① Create a clone list (using next ptr of original list) $\rightarrow O(n)$

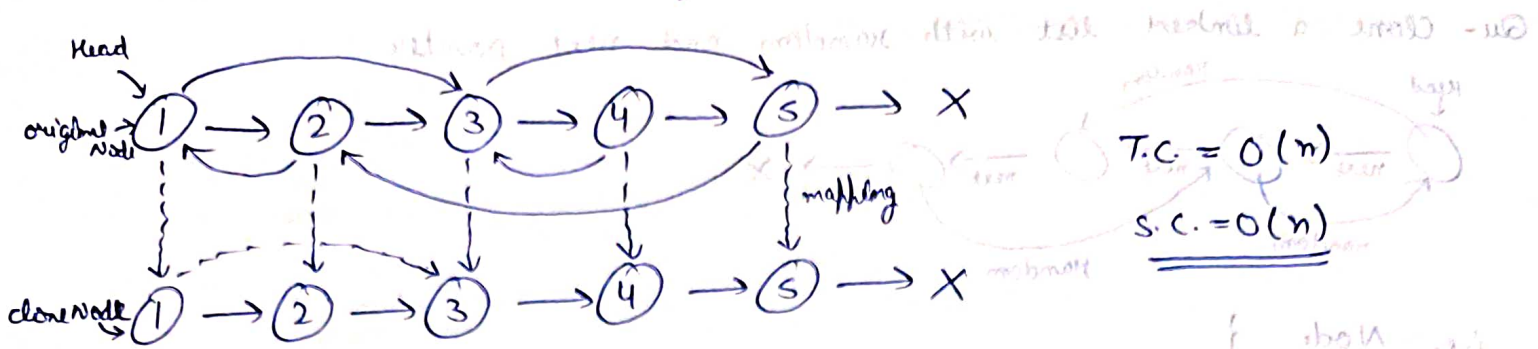


② mapping



$\Rightarrow \text{cloneNode} \rightarrow \text{random} = \text{mapping} [\text{originalNode} \rightarrow \text{random}]$

Q3 solution



```
void insertAtTail (Node* & head, Node* & tail, int d) {
    Node* newNode = new Node(d);
    if (head == NULL) {
        head = newNode;
        tail = newNode;
        return;
    }
    tail->next = newNode;
    tail = newNode;
}
```

}

else {

tail->next = newNode;

tail = newNode;

}

}

Node* copyList (Node* head) {

Node* cloneHead = NULL;

Node* cloneTail = NULL;

Node* temp = head;

while (temp != NULL) {

insertAtTail (cloneHead,

cloneTail, temp->data);

temp = temp->next;

}

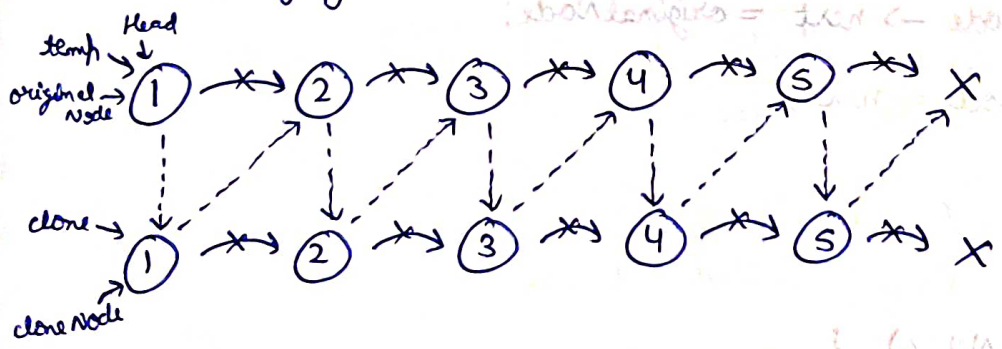
```

unordered_map < Node*, Node* > oldToNewNode;
Node* originalNode = head;
Node* cloneNode = cloneHead;
while (originalNode != NULL && cloneNode != NULL) {
    oldToNewNode[originalNode] = cloneNode;
    originalNode = originalNode -> next;
    cloneNode = cloneNode -> next;
}
originalNode = head;
cloneNode = cloneHead;
while (originalNode != NULL) {
    cloneNode -> random = oldToNewNode[originalNode -> random];
    originalNode = originalNode -> next;
    cloneNode = cloneNode -> next;
}
return cloneHead;

```

Approach 3 → (T.C. = O(n), S.C. = O(1)) i.e. map use nhi krna

i.e. changing links



① Create a clone list

② cloneNode add in b/w original list

③ Random pointer copy

⇒ temp → next → random = temp → random → next;

④ revert changes done in step ②

⑤ return ans (cloneHead)

Node* copyList (Node* head) {

Node* cloneHead = NULL;

Node* cloneTail = NULL;

Node* temp = head;

while (temp != NULL) {

insertAtTail (cloneHead, cloneTail, temp → data);

temp = temp → next;

}

Node* originalNode = head;

Node* cloneNode = cloneHead;

while (originalNode != NULL && cloneNode != NULL) {

Node* next = originalNode → next;

originalNode → next = cloneNode;

originalNode = next;

next = cloneNode → next;

cloneNode → next = originalNode;

cloneNode = next;

}

temp = head;

while (temp != NULL) {

if (temp → next != NULL) {

temp → next → random = temp → random → next;

}

temp = temp → next;



temp = temp → next → next;

}

originalNode = head;

cloneNode = cloneHead;

while (originalNode != NULL & cloneNode != NULL) {

originalNode → next = cloneNode → next;

originalNode = originalNode → next;

if (originalNode != NULL) {

cloneNode → next = originalNode → next;

}

cloneNode = cloneNode → next;

}

return cloneHead;

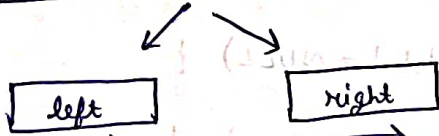
}

Lecture 53

Q- Sort the linked list using merge sort algorithm

Approach

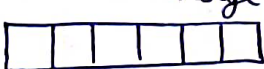
in case of arrays



sort

sort

Merge



T.C. $\Rightarrow O(n \log n)$

Same logic will be for linked list

Node* findMid(Node* head) {

Node* slow = head;

Node* fast = head->next;

while (fast != NULL && fast->next != NULL) {

slow = slow->next;

fast = fast->next->next;

}

return slow;

}

Node* merge(Node* left, Node* right) {

if (left == NULL) {

return right;

}

if (right == NULL) {

return left;

}

Node* ans = new Node(-1);

Node* temp = ans;

while (left != NULL && right != NULL) {

if (left->data < right->data) {

temp->next = left;

temp = left;

left = left->next;

}

else {



temp₁ → next = right;

temp₁ = right;

right = right → next;

}

}

while (left != NULL) {

temp → next = left;

temp = left;

left = left → next;

}

while (right != NULL) {

temp → next = right;

temp = right;

right = right → next;

}

ans = ans → next;

return ans;

}

Node* mergeSort (Node* head) {

if (head == NULL || head → next == NULL) {

return head;

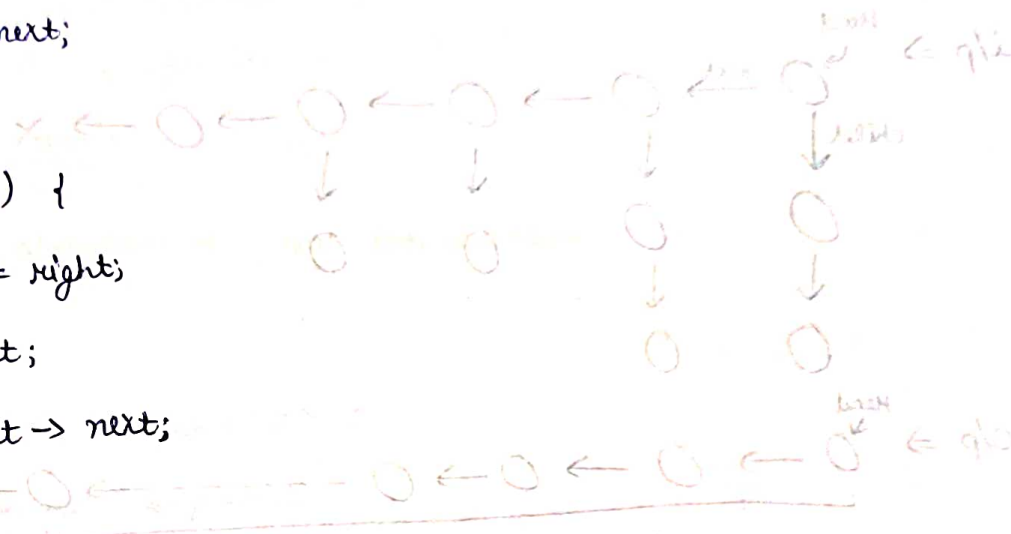
}

Node* mid = findMid (head);

Node* left = head;

Node* right = mid → next;

mid → next = NULL;



```

left = mergesort(left);
right = mergesort(right);

```

```

Node * result = merge(left, right);

```

```

return result;

```

```

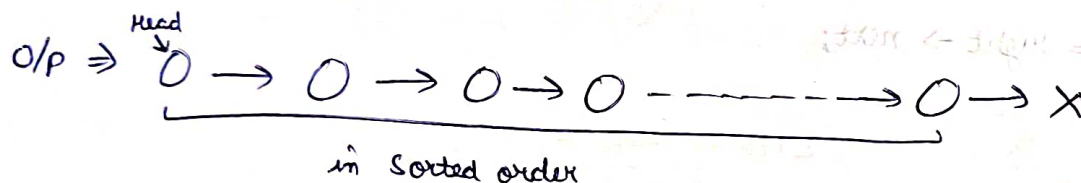
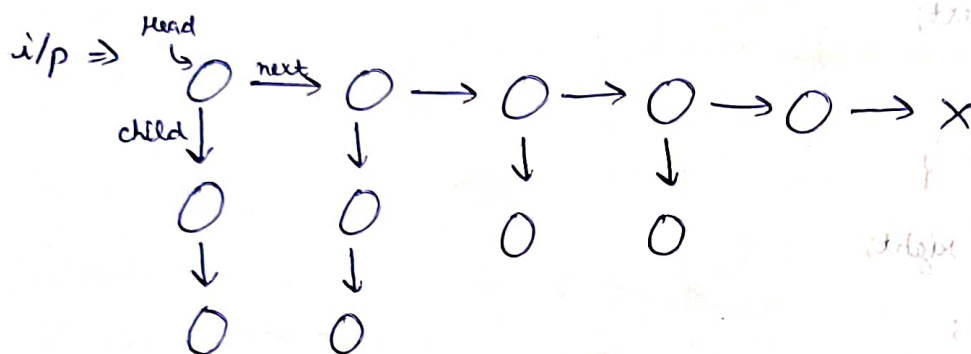
}

```

H.W. Question

Why prefer merge sort in linked list and quick sort in arrays

Q- Flatten a linked list



Approach

↳ By Recursion

```

flatten(head) {

```

```

Node* down = head;

```

```

down->next = NULL;

```

```

Node* right = flatten(head->right);

```

```

Node* ans = merge(down, right);

```

```

return ans;

```

```

}

```

H.W.

Code the above problem